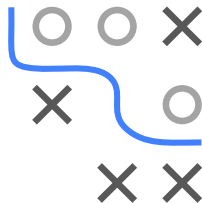


Optimization in Machine Learning

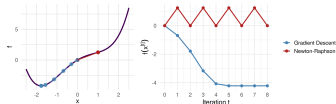
Second order methods

Limitations of Newton-Raphson



Learning goals

- Non-descent directions
- Singular and indefinite Hessian
- Saddle points
- Computational cost



Slides based on

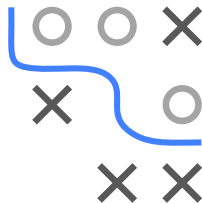
► Aggarwal 2020 (Ch. 5.4-5.7)

LIMITATIONS: NON-DESCENT DIRECTIONS

- **Problem 1:** In general, $\mathbf{d}^{[t]}$ is not a descent direction
- **But:** If Hessian is positive definite, $\mathbf{d}^{[t]}$ is descent direction:

$$\nabla f(\mathbf{x}^{[t]})^\top \mathbf{d}^{[t]} = -\nabla f(\mathbf{x}^{[t]})^\top (\nabla^2 f(\mathbf{x}^{[t]}))^{-1} \nabla f(\mathbf{x}^{[t]}) < 0$$

- Near minimum, Hessian is positive definite
- For initial steps, Hessian is often not positive definite and Newton-Raphson may give non-descending update directions

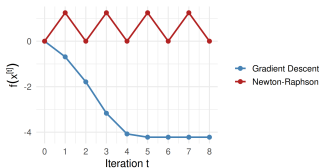
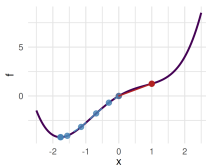
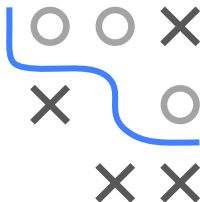


EXAMPLE: NON-DESCENT DIRECTIONS

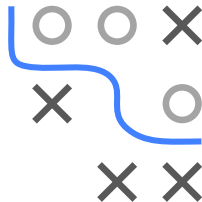
- Consider $f(x) = \frac{x^4}{4} - x^2 + 2x$ with $f'(x) = x^3 - 2x + 2$, $f''(x) = 3x^2 - 2$
- Starting at $x^{[0]} = 0$: $f''(0) = -2 < 0$ (**H** not pd) $\Rightarrow$ Newton-Raphson performs **ascent** step:

$$x^{[1]} = x^{[0]} - \frac{f'(x^{[0]})}{f''(x^{[0]})} = 0 - \frac{2}{-2} = 1, \quad f(x^{[1]}) = 1.25 > 0 = f(x^{[0]})$$

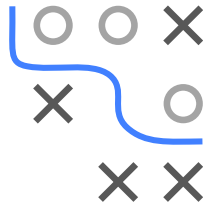
- From $x^{[1]} = 1$: $f''(1) = 1 > 0$, but the step leads right back to $x^{[2]} = 0$
 $\Rightarrow$ Newton-Raphson cycles forever between 0 and 1
- GD** from the same $x^{[0]} = 0$ only uses the neg. gradient $-f'(x^{[0]}) = -2$ (descent direction) and converges for appropriate step size α



LIMITATIONS: SINGULAR AND INDEFINITE HESSIAN



LIMITATIONS: SADDLE POINTS



LIMITATIONS: COMPUTATIONAL COST

- **Problem 4:** Hessian can be **computationally expensive** to calculate, as descent direction $\mathbf{d}^{[t]}$ is solution of the linear system

$$\nabla^2 f(\mathbf{x}^{[t]})\mathbf{d}^{[t]} = -\nabla f(\mathbf{x}^{[t]})$$

- For $\mathbf{x} \in \mathbb{R}^d$, the Hessian has d^2 entries: $\mathcal{O}(d^2)$ memory, and $\mathcal{O}(d^3)$ per iteration to solve the linear system (compared to $\mathcal{O}(d)$ memory and no linear solve for GD)
- For large d (e.g., deep neural networks with $d \sim 10^9$ parameters), this is prohibitive: storing a dense $d \times d$ Hessian alone is infeasible
- **Aim:** Find quasi-second order methods not relying on exact Hessians
 - Quasi-Newton method
 - Gauss-Newton algorithm (for least squares)

